Nom de la revue / Conférence

Titre principal de l'article

Sous-titre ou précision si nécessaire

Prénom Nom[1] **Prénom Nom[2]**

[1]Laboratoire / Institution, Ville, Pays

[2]Laboratoire / Institution, Ville, Pays

auteur@institution.fr

Soumis le : 27 avril 2026 | Accepté le : JJ/MM/AAAA

Résumé

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1. Introduction

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2. Analyse du code initial

Le code original se structure comme suit:

1. Une phase d'initialisation.
2. Envoi des cellules fantômes.
3. Calcul de la vitesse des particules fluides et de la densité de ce dernier en tout points.
4. Actualisation de la position du fluide.
5. Envoi des cellules fantômes.
6. ...

Par conséquent, nous pouvons extraire 3 grands axes d'optimisation :

2.1. Communication entre processus MPI

Dans cette section, nous allons travailler sur plusieurs axes. Le premier sera sur la communication en elle même. En effet, nous allons chercher à privilégier les communications non bloquantes quand c'est possible, s'assurer que le maillage et la répartition des tâches sont optimales en les comparant avec l'état de l'art pour ce genre de problème, comparer s'il est plus judicieux de faire des communications collectives ou de point à point, limiter les synchronisations entre processus pour des calculs plus fluides.

2.2. Accès mémoire et optimisations de caches

Les calculs étant juste pour cette première version du simulateur, notre objectif dans cette section va être de se concentrer sur la manière dont les calculs sont réalisés. Ainsi, nous chercherons à optimiser la localité mémoire (spatiale et temporelle); à s'assurer que l'allocation mémoire et la déclaration de variables soient pertinentes et à vérifier d'éventuels effets NUMA.

2.3. Micro-optimisations de calculs inutiles ou bugs éventuels

En fonction de notre analyse de la structure de certains calculs, il est possible de rencontrer certaines parties qui sont inutiles pour l'obtention du résultat final, tels que des valeurs étant réécrites sans utilisation particulière, des fonctions non-utilisées ou encore une structure de code n'assurant pas l'obtention de bugs dans tout les cas possibles. Ainsi, lorsque nous rencontrons une micro-optimisation réalisable, nous la mettons en place sans toujours l'avoir repérée depuis un profiler.

3. Optimisations apportées

Nous utiliserons plusieurs outils de profiling pour déceler les points chauds du programme:

- *Instruments.app*: Flame Graph, nombre d'instructions et de cycles CPU, IPC (Instructions per cycles) moyens

Tout graphique et résultats d'outils de profiling seront mis en annexes.

3.1. Première approche

Afin de commencer à chercher les premières optimisations, nous avons tracer un Flame graph du programme (cf Annexe A). Nous pouvons ainsi nous rendre compte que les communications MPI pour un maillage en halo sont le point chaud majeur, et ralentissent significativement le programme. C'est donc la première chose sur laquelle nous allons nous concentrer.

3.2. Optimisation des communications MPI

La première de ces optimisations est le fait de retirer le 2e envoi de maille fantôme vers la gauche, puisqu'il n'est pas nécessaire de le faire 2 fois.

Nous constatons que les performances sont réduites à cause de 3 fonctions MPI: *MPI_Barrier*, *MPI_Send* et *MPI_Recv* dans la fonction *lbm_comm_halo_exchange* (cf Annexe B). Nous allons donc repenser comment les mailles fantômes sont envoyées.
Les *MPI_Barrier* sont un problème majeur dans cette fonction. En effet, il est inutile de mettre des MPI_Barrier ici, puisque le buffer envoyé est différent pour chacune des fonctions d'envoi et les communications étant bloquante, la synchronisation est déjà incluse.
Comme dit précédemment, les buffers envoyés par communication sont différents (cf. Figure 1).

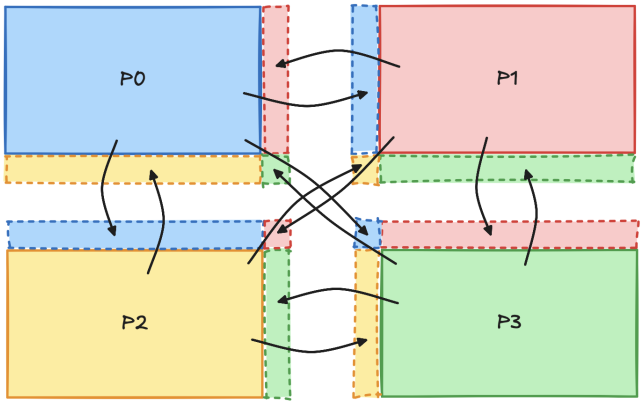


FIGURE 1 – Halo exchange communication scheme for 4 MPI processes

Ainsi, nous pouvons utiliser des communications non-bloquantes sur chacune des fonctions d'envoi de mailles à l'aide de *MPI_Isend* et *MPI_Irecv*, avec un *MPI_Waitall* à la place de la série de *MPI_Barrier*, pour s'assurer que le programme calcul sur une zone qui n'a pas encore été complètement reçue puis avons ajouter un tag unique en fonction du rang et de la direction de la communication afin que les communications soient déterministes. Ceci sera notre première optimisation.

Nous avons également remarquer d'autres *MPI_Barrier* au sein même de la boucle de calcul entre chaque étape, forçant les processus MPI à se synchroniser après le calcul des collisions, de l'échange de mailles fantômes et du calcul de la propagation. Or, cela est sans justification car les processus ont besoin d'être synchronisés uniquement lors de l'envoi de message, pour éviter d'entamer le calcul sur des buffers qui ne sont pas encore pleinement envoyés. Nous pouvons ainsi supprimer l'entièreté des *MPI_Barrier*.

Nous avons remarquer que la communication "Right to left" était dupliquée, nous avons donc simplement supprimer la seconde communication. Notons donc que nous avons 8 échanges point à point par appel à la fonction *lbm_comm_halo_exchange*, soit que le *MPI_Waitall* attend 16 requests à la fin de son exécution.

3.3. Optimisation mémoire

Maintenant que les communications ont été optimisées, nous allons nous pencher sur l'accès à la mémoire, pour ce faire, nous avons utiliser le profiler perf, en évaluant les

- instructions
- cycles
- cache-references
- cache-misses

Nous avons comme résultat un total de 32 Million cache-references et 17,920 Million cache-misses soit près de la moitié de la recherche de la mémoire n'est pas directement accedée, et doit être cherchée dans un espace d'adressage plus distant. En nous penchant sur les fonctions ne montrant pas une bonne gestion de la mémoire, nous trouvons :

- *global data* 934 B instructions, 16 B cache-misses
- *compute_cell_collision* comporte 51.41% des instructions et 37.71% des cache-misses;
- *propagation* comporte 21.64% des instructions et 58.33% des cache-misses.

Ces observations sont dûes aux tableaux Mesh::cells qui sont de taille : $width \times height \times directions \times sizeof(double)$, soit : $800*160*9*8 = 9,216,000B$ ce qui est supérieur à l'ensemble du cache L3 sur lequel perf à été exécuté, L3 : $8,192,000B$. En effet, ces deux fonctions appels *Mesh_get_cell* qui charge en mémoire la cellule stockée dans le tableau. Ce nombre de cache-miss est naturellement élevé, mais ces fonctions peuvent tout de même être optimisée. Commençons par *compute_cell_collision* :

Elle est appelée par *collision* qui charge d'abord deux cellules dans une double boucle for, suivi de ces lignes :

```
const double density = get_cell_density(cell_in);
Vector v;
get_cell_velocity(v, cell_in, density);
double v2 = get_vect_norm_2(v, v);
```

Ces valeurs ne dépendent que de "cell_in" et peuvent être initialisées en dehors de la double boucle. Nous faisons donc ainsi, les calculons avant l'appel à la fonction puis les passons en arguments const.

Ensuite notre fonction réalise une troisième boucle sur les 9 dimensions afin d'actualiser "cell_out" en fonction de "f_eq" et "cell_in", nous sortons également la déclaration de "f_eq" des boucles for et le passons en argument, non const. Cela allège les performances notées par perf de la fonction *collision* puisque nous nous assurons de réutiliser les variables deja initialisées en cache, sans avoir a allouer à allouer de nouvelles variables et donc d'évincer plus de mémoire pour chaque itération des trois boucles. Donnant par la suite :

- *global data* 824 B instructions, 16 B cache-misses
- *collision* comporte 52.54% des instructions et 37.91% des cache-misses;

prof version: 54 107 M cache-misses ; 311 945 M cache-references collision : 7.08% cache-misses ; 8.37% -> 14.67$ propagation : 11.41% cache-misses ; 9.79% -> 20.21%

My version: 4 146 M cache-misses ; 50 758 M cache-references collision : 11.46% cache-misses ; 22.23% -> 4.21% propagation:33.26% cache-misses ; 35.85% -> 7.57%

4. Expériences

4.1. Protocole expérimental

Notre approche pour vérifier les optimisations que nous ferons est la suivante: pour chaque optimisation, nous mesurerons la variation de fréquence de mise à jour des cellules (en MLUPS) et vérifierons que les résultats restent vrais en les comparant avec une référence.

4.1.1. Mesure du nombre d'opérations par secondes

Le programme intègre déjà une mesure en MLUPS (Million Lattice Updates Per Second), déterminée comme suit:

$$Mlups = \frac{total_cells \cdot ITERATIONS}{elapsed_time}$$

avec

- *total_cells*: le nombre total de cellules dans le maillage
- *ITERATIONS*: le nombre total de mises à jour faites par le programme
- *elapsed_time*: le runtime de la boucle de calcul

Nous mesurerons cette quantité 3 fois, puis nous analyserons la différence avec la précédente:

$$Mlups_{final} = \sum_{i=1}^{3} \frac{\left|Mlups^{precedent} - Mlups_i^{mesure}\right|}{3}$$

Les incertitudes seront déterminées par la déviations standard de nos valeurs mesurées.

4.1.2. Vérification des résultats

Afin de d'assurer la cohérence de notre modèle suite aux optimisations apportées, nous quantifierons celles-ci en analysant l'écart entre les données de référence (cf. Table 1) et celles mesurées.
Pour ce faire, nous utiliserons le script *Python* fourni qui consiste à sommer toutes les valeurs de densité ρ et la norme du vecteur vitesse $U(\mathbf{x}, t)$ de toutes les cellules pour chaque frame calculée du jeu de donnée de référence ainsi que pour chaque frame après optimisation, avant de comparer les résultats. La solution étant déterministe, les résultats devront être identiques.

Table 1 – Configuration de référence

Paramètres	**Valeurs**
Taille de la conduite	800×160
Position de l'obstacle	$\{100, 80\}$
Rayon de l'obstacle	0.16
Nombre de Reynolds de l'écoulement Re	96
Vitesse maximal du fluide $U(x = 0, y, t)$	0.16

Ainsi, nous lancerons les commandes suivantes:

```
uv venv
uv run lbm-viz --check\
 ref_results.raw results.raw
```

Les mesures seront fait avec les flags d'optimisations *-O3* et *-march=native*. Le compilateur utilisé sera indiqué pour chaque série de mesures.

4.2. Résultats

Voici un tableau récapitulatif des optimisations et leurs impacts sur le Mlups (cf Table 2), la valeur Mlups mesurée pour la version initale est $Mlups = 125 \pm 3$:

Table 2 – Optimisation de la communication dans l'halo

Optimisation	**Mlups**	δ_{Mlups}
Comm. non-bloquantes	1575	± 20
Sup. *MPI_Barrier* main	2363	± 20

Table 3 – Optimisation de la mémoire

Optimisation	**Mlups**	δ_{Mlups}

5. Résultats

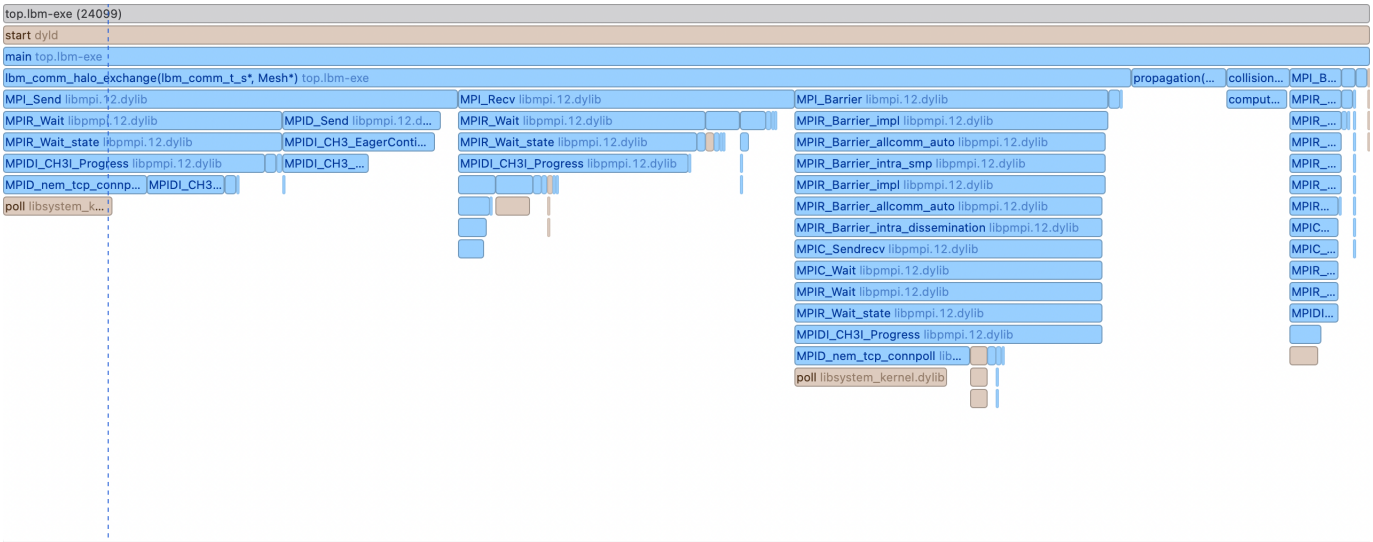
Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

6. Conclusion

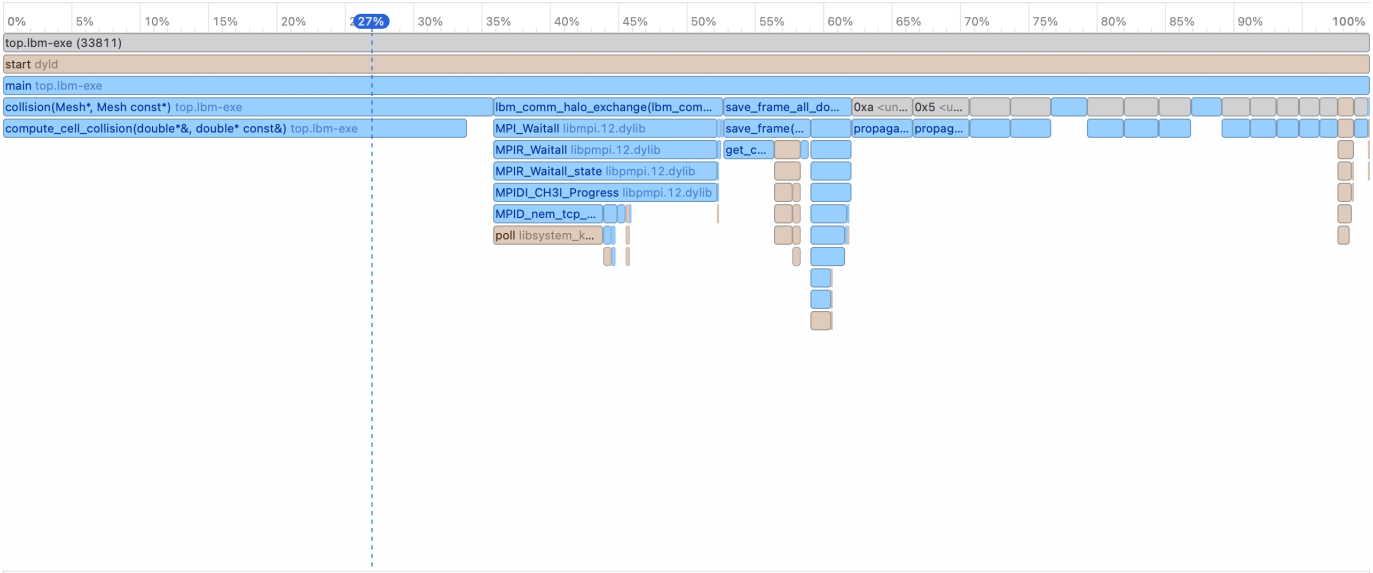
Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Annexes

A. Flame Graph initial: Instruments.app



B. Fonction *lbm_comm_halo_exchange* initiale

```
void lbm_comm_halo_exchange(lbm_comm_t* mesh, Mesh* mesh_to_process) {
  int rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_SEND, mesh->right_id, mesh->width - 2);
  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_RECV, mesh->left_id, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_SEND, mesh->left_id, 1);
  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_RECV, mesh->right_id, mesh->width - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_SEND, mesh->bottom_id, mesh->height - 2);
  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_RECV, mesh->top_id, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_SEND, mesh->top_id, 1);
  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_RECV, mesh->bottom_id, mesh->height - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_TOP_LEFT], 1, 1);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_BOTTOM_RIGHT], mesh->width - 1,
mesh->height - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_BOTTOM_LEFT], 1, mesh->height - 2);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_TOP_RIGHT], mesh->width - 1, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_TOP_RIGHT], mesh->width - 2, 1);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_BOTTOM_LEFT], 0, mesh->height - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_BOTTOM_LEFT], 1, mesh->height - 2);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_TOP_RIGHT], mesh->width - 1, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_BOTTOM_RIGHT], mesh->width - 2, mesh->height - 2);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_TOP_LEFT], 0, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_SEND, mesh->left_id, 1);
  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_RECV, mesh->right_id, mesh->width - 1);

  MPI_Syncall(MPI_COMM_WORLD);
}
```

C. Flame Graph après optimisation MPI: Instruments.app